7 月 5 日学习总结

课堂内容

一，一维数组应用题

二，上机编写程序实战

三，数组题目讲解

四，二维数组及题目

五，课下作业

一维数组应用

题目描述

最近小明想用计算器计算100位数的加法运算，你能用所学的数组知识帮他解决这个问题吗?

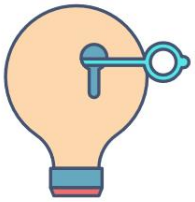
为了降低题目难度，计算器的两个操作数位数相同（<101）。

输入

1234321234321234321 2
4321234321234321234 3

输出

55555555555555555555



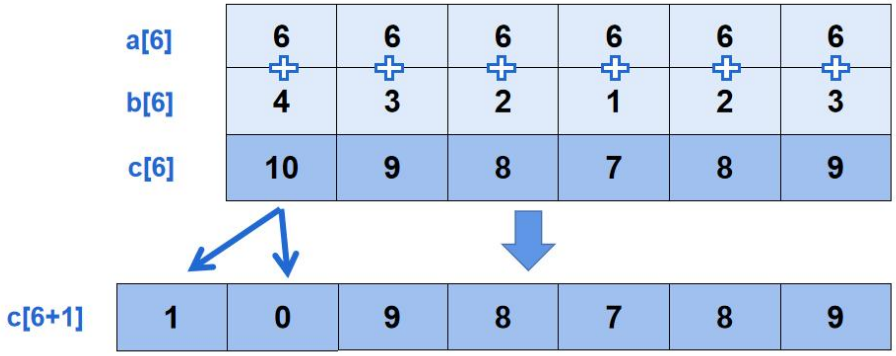
一维数组应用

如何用数组来存入操作数

a[6]	1	2	3	4	3	2
	+	+	+	+	+	+
b[6]	4	3	2	1	2	3
c[6]	5	5	5	5	5	5

一维数组应用

如果产生进位该怎么办呢



一维数组应用

从i+1开始赋值，将下标为0的位置预留（防止最高位进位时溢出）

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
        char a[100],b[100];
        int c[101]={0};
        cin>>a>>b;
        for(int i=strlen(a)-1;i>=0;i--)
        {
                c[i+1]+=a[i]+b[i]-96;
                if(c[i+1]>9) c[i]+=c[i+1]/10,c[i+1]%=10;
        }
        if(c[0]) cout<<c[0];
        for(int i=1;i<=strlen(a);i++) cout<<c[i];
        return 0;
}
```

一维数组应用

题目描述

小红最近在学习排序，做题时想用程序很快检查自己做的题是否正确，你能帮忙解决这个问题吗？

输入

5
5 3 1 2 4

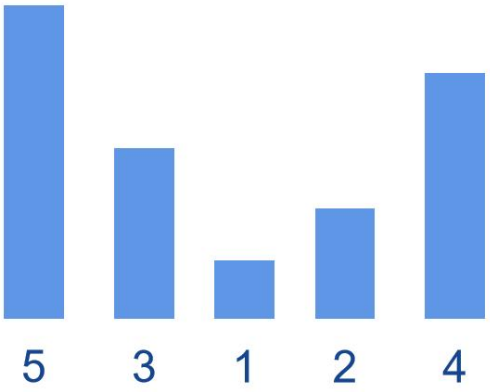
输出

1 2 3 4 5

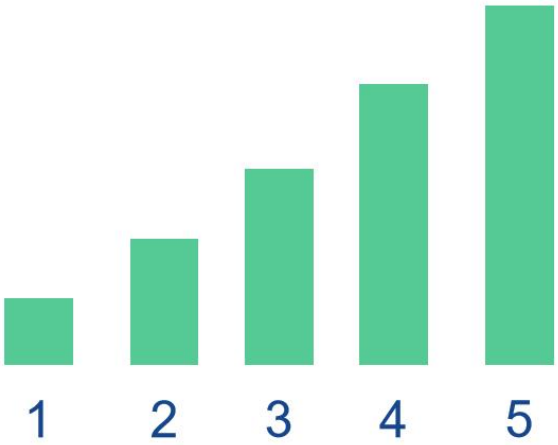


一维数组应用

大家想一想有哪些办法能够将
5 3 1 2 4 从小到大排序



一维数组应用



一维数组应用

刚刚的排序思路就是
著名的冒泡排序

i,j总是从0,1开始
len的大小一直在减小

```
int n,a[100];
cin>>n;
for(int i=0;i<n;i++) cin>>a[i];
for(int i,j,len=n;len>0;len--)
{
        for(i=0,j=1;j<len,i<len-1;i++,j++)
        {
                if(a[i]>a[j])
                {
                        int temp = a[i];
                        a[i] = a[j];
                        a[j] = temp;
                }
        }
}
for(int i=0;i<n;i++) cout<<a[i]<<" ";
return 0;
```

一维数组应用

还有一个选择排序的
写法是这样的

```
for(int i=0;i<n;i++)
{
        for(int j=i+1;j<n;j++)
        {
                if(a[i]>a[j])
                {
                        int temp = a[i];
                        a[i] = a[j];
                        a[j] = temp;
                }
        }
}
```

一维数组应用

两层for循环，实现
两数的比较

```
for(int i=0;i<n;i++)
{
        for(int j=i+1;j<n;j++)
        {
                if(a[i]>a[j])
                {
                        int temp;
                        temp = a[i];
                        a[i] = a[j];
                        a[j] = temp;
                }
        }
}
```

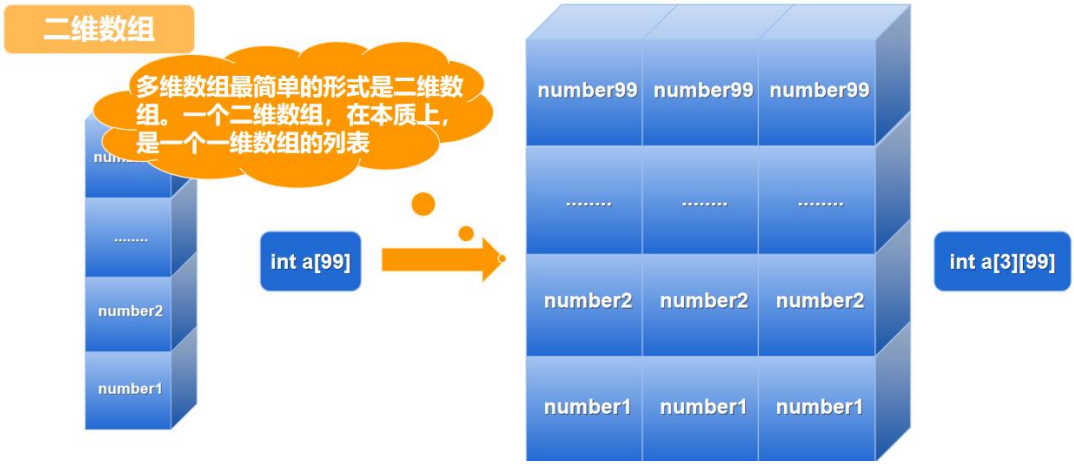
一维数组应用

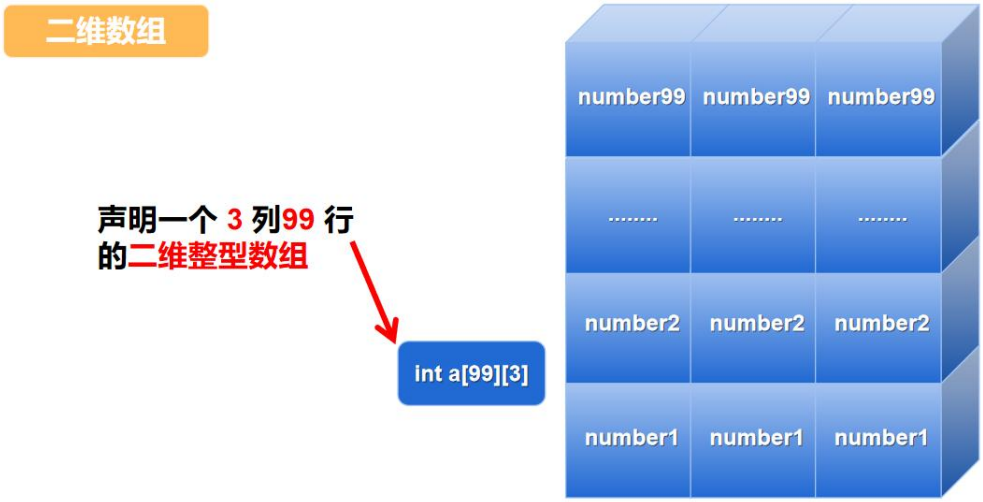
如果a[i]比a[j]大，
则交换两数

```
for(int i=0;i<n;i++)
{
        for(int j=i+1;j<n;j++)
        {
                if(a[i]>a[j])
                {
                        int temp = a[i];
                        a[i] = a[j];
                        a[j] = temp;
                }
        }
}
```

总结

- 用数组解决实际问题
- 数组的排序





二维数组

其中，type 可以是任意有效的 C++ 数据类型，arrayName 是一个有效的 C++ 标识符。

一个二维数组可以被认为是一个带有 x 行和 y 列的表格。右侧是一个二维数组，包含 3 行和 4 列

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

数组中的每个元素是使用形式为 a[i , j] 的元素名称来标识的，其中 a 是数组名称，i 和 j 是唯一标识 a 中每个元素的下标

初始化二维数组

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

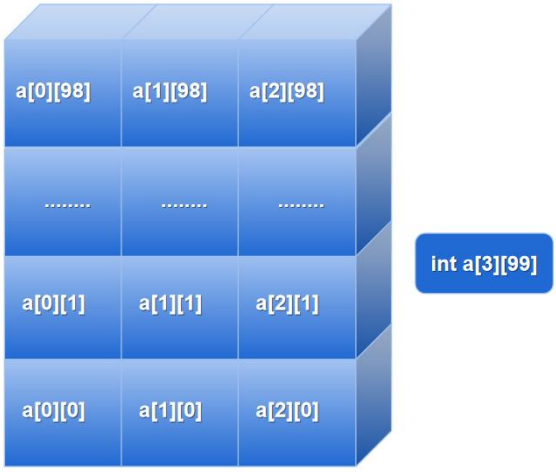
内部嵌套的括号是可选的

```
int a[3][4] = {
 {0, 1, 2, 3} ,   /*  初始化索引号为 0 的行 */
 {4, 5, 6, 7} ,   /*  初始化索引号为 1 的行 */
 {8, 9, 10, 11}   /*  初始化索引号为 2 的行 */
};
```

访问二维数组元素

二维数组中的元素是通过使用下标（即数组的行索引和列索引）来访问的

a[0][1]



二维数组

```
a[0][0]: 0
a[0][1]: 0
a[1][0]: 1
a[1][1]: 2
a[2][0]: 2
a[2][1]: 4
a[3][0]: 3
a[3][1]: 6
a[4][0]: 4
a[4][1]: 8
```

第一轮循环遍历行

第二轮循环遍历列

五行，最后一行下标为5-1

```
#include <iostream>
using namespace std;
int main ()
{
  // 一个带有 5 行 2 列的数组
  int a[5][2] = { {0,0}, {1,2}, {2,4},
{3,6},{4,8}};
  // 输出数组中每个元素的值
  for ( int i = 0; i < 5; i++ )
    for ( int j = 0; j < 2; j++ )
    {
      cout << "a[" << i << "][" << j << "]: ";
      cout << a[i][j]<< endl;
    }
  return 0;
}
```

二维数组实战

题目描述

某校大门外长度为 l 的马路上有一排树，每两棵相邻的树之间的间隔都是 1米。我们可以把马路看成一个数轴，马路的一端在数轴 0 的位置，另一端在 l的位置；数轴上的每个整数点，即 0,1,2,…,l，都种有一棵树。

由于马路上有一些区域要用来建地铁。这些区域用它们在数轴上的起始点和终止点表示。已知任一区域的起始点和终止点的坐标都是整数，区域之间可能有重合的部分。现在要把这些区域中的树（包括区域端点处的两棵树）移走。你的任务是计算将这些树都移走后，马路上还有多少棵树。

输入格式

第一行有两个整数，分别表示马路的长度 l 和区域的数目 m。

接下来 m 行，每行两个整数 u, v，表示一个区域的起始点和终止点的坐标。

输出格式

输出一行一个整数，表示将这些树都移走后，马路上剩余的树木数量。

输入：

```
500 3
150 300
100 200
470 471
```

输出：

```
298
```

二维数组实战

将第i次操作中的第a[i][0]到a[i][1]的树的状态改为1

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
        int a[100][2],l,m,b[10000]={0},count=0;
        cin>>l>>m;
        for(int i=0;i<m;i++) cin>>a[i][0]>>a[i][1];
        for(int i=0;i<m;i++)
                for(int j=a[i][0];j<=a[i][1];j++)
                        b[j]=1;
        for(int i=0;i<=l;i++)
                if(b[i]==0) count++;
        cout<<count;
        return 0;
}
```

二维数组实战

题目描述

给出 n(n≤20)，输出杨辉三角的前 n 行。

如果你不知道什么是杨辉三角，可以观察样例找找规律。

输入

6

输出

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1

二维数组实战

数值中杨辉三角形外的数已初始化为0，切未参与计算，所以不输出

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
        int n,a[22][22]={0};
        cin>>n;
        a[1][1]=1;
        for(int i=1;i<=n;i++)
                for(int j=1;j<=n;j++)
                        a[i][j] += a[i-1][j] + a[i-1][j-1];
        for(int i=1;i<=n;i++)
        {
                for(int j=1;j<=n;j++)
                        if(a[i][j]!=0)
                                cout<<a[i][j]<<" ";
                cout<<"\n";
        }
        return 0;
}
```

课后作业:

1，一维数组和二维数组及应用题(详见课堂笔记本所记相关笔记)。

2，杨辉三角形，详见课堂笔记和经典例题上机实践